Chapter 10 Eligibility Traces

资格迹 (Eligibility Traces)

资格迹是强化学习的基本机制之一。它可以与任何**时序差分 (TD)** 方法（例如 Sarsa 和 Q-learning）结合使用，从而提高学习效率。
资格迹提供了一种**统一并泛化 TD 方法和蒙特卡洛 (Monte Carlo, MC) 方法**的途径。具体来说，资格迹引入了一个**迹衰减参数 (trace-decay parameter)** $\lambda \in [0, 1]$。通过调节这个参数，可以得到一个算法谱系。

$\lambda = 0$ 对应单步 TD 方法，$\lambda = 1$ 对应蒙特卡洛方法。

此外，资格迹还为强化学习算法带来了一个新的视角，称为**后向视图 (backward views)**，这使得在线 (online) 算法的实现更加高效。

想象你正在打通关游戏。你在第1秒做了一个“向左闪避”的动作，第10秒做了一个“跳跃”的动作，然后在第15秒你击败了Boss，获得了 +100 的奖励。

- **信用分配问题**：这 +100 分的奖励，到底是归功于第1秒的闪避，还是第10秒的跳跃？
- **后向视图与资格迹**：当你在第15秒拿到奖励时，你“往回看”（后向视图）。你脑海中对刚刚做过的动作会有一个“记忆痕迹”（这就是**迹 Trace**）。距离现在越近的动作（跳跃），记忆越清晰，分到的功劳（**资格 Eligibility**）就越多；距离越远的动作（闪避），记忆衰减了，分到的功劳就少。参数 λ 就是用来控制这个“记忆衰减速度”的。

λ-回报 (The λ-return)

我们要先回顾一下 n-**步回报 (**n**-step return)**。它被定义为前 n 步的真实奖励之和，加上 n 步之后状态的估计价值。

公式 (10.1) - n **步回报：**

$$g_{t:t+n} = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{n-1} r_{t+n} + \gamma^n \hat{v}(s_{t+n}, \mathbf{w}_{t+n-1})$$

(前提：$0 \le t \le T - n$，其中 T 是回合结束时间。对于持续性任务，没有终止状态。)
在 n-步 TD 方法中，$g_{t:t+n}$ 被用作**更新目标 (update target)**。

我们还可以将更新目标进一步泛化，变成不同 n 步回报的**平均值**。例如，把 2 步回报和 4 步回报各取一半 $(g_{t:t+2} + g_{t:t+4})/2$，这也是一个合法的更新目标。

λ-回报 (λ-return) 是一种特定的平均 n-步回报的方法。它将**所有**的 n-步回报组合起来，每个 n-步回报的权重与 λ^{n-1} 成正比。

公式 (10.2) - λ 回报的定义：

$$g_t^\lambda = (1-\lambda)\sum_{n=1}^{\infty}\lambda^{n-1}g_{t:t+n}$$

- $g_{t:t+n}$：从时间 t 往后看 n 步的累积回报。
- r_{t+1}：时间 $t+1$ 瞬间收到的真实奖励。
- γ：折扣因子 (Discount factor)，范围 $[0,1]$。代表未来奖励在今天的价值折损。
- $\hat{v}(s,\mathbf{w})$：状态价值函数。意思是，基于当前的权重参数 $\mathbf{w}$，我“猜”处于状态 s 以后还能拿多少分。
- g_t^λ：这是一种终极的、综合了所有的回报估算方法的目标值。
- $(1-\lambda)$：这是一个归一化系数，为了保证所有的权重加起来正好等于 1。

λ^{n-1} 在这里代表的是**分配给第 n 步回报的“权重（Weight）” 注意为n-1为幂，不是角标！**

为什么要累加（$\sum_{n=1}^{\infty}$）？
累加的本质是在做一个**加权平均（Weighted Average）**。强化学习中面临一个经典的困境：**偏差与方差的权衡（Bias-Variance Trade-off）**。
• **如果只看 1 步（$\mathrm{TD}(0)$）**：估计结果方差小（因为基于眼前的真实奖励），但是偏差大（因为高度依赖对未来的猜测）。
• **如果看无穷步（蒙特卡洛方法，一直走到回合结束）**：没有任何猜测，偏差为 0，但是方差极大（因为路径太长，充满了各种随机性）。
累加的巧妙之处在于，它不单独依赖任何一种步长，而是“全都要”。 它把从 1 步、2 步、3 步一直到无穷步（或回合结束）的所有回报结果，按照前面提到的权重 λ^{n-1} 全部加起来，从而完美融合了短期估计和长期实际结果的优点。
补充一个关键细节：前面的 $(1-\lambda)$ 是干嘛的？
既然是做加权平均，所有权重加起来必须等于 1。
根据无穷等比数列的求和公式，所有权重的和是：

$$\sum_{n=1}^{\infty} \lambda^{n-1} = 1 + \lambda + \lambda^2 + \cdots = \frac{1}{1-\lambda}$$

为了让权重总和为 1，我们在求和符号前面乘以 $(1-\lambda)$，这样就构成了一个完美的加权平均数。

为什么要“平均”各种 n-步回报？

- **单步 TD (走一步看一步)**：就像天气预报只预测明天，然后根据明天的实际天气来调整预测模型。优点是反应快，缺点是太短视。
- **蒙特卡洛 (走到大结局)**：就像预测一整年的气候，必须等这一年过完才能验证。优点是绝对准确（没有猜测成分），缺点是等太久，方差大（中间变数太多）。
- **λ-回报 (我全都要)**：与其纠结到底是看 1 步、2 步还是 10 步，不如把所有的预测都算出来！单步回报占一点权重，2步回报占一点权重，一直到无限步。λ 就是分配这些权重的“比例尺”。（ ，权重呈现指数衰减）。

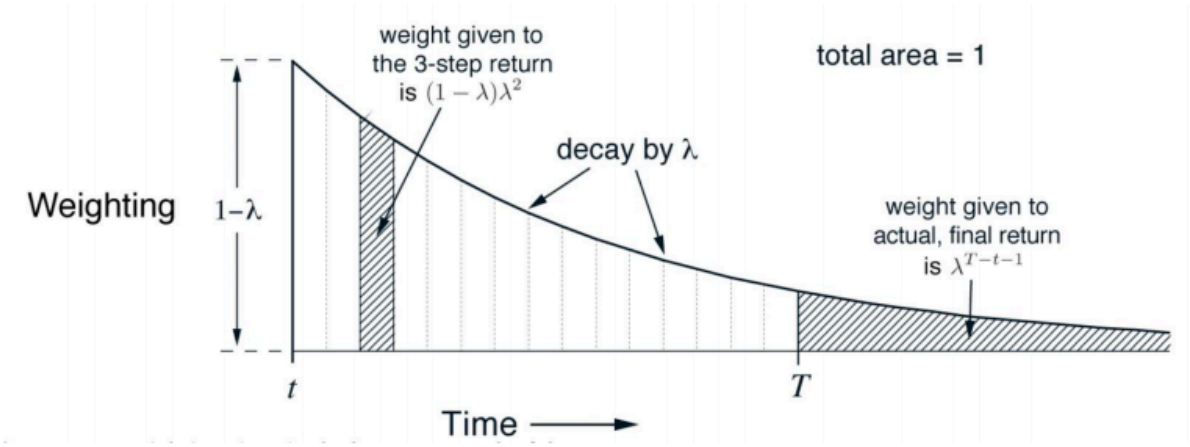

Figure 10.1: The distribution of weights in the λ-return.

幕式任务 (Episodic Tasks) 中的 λ-回报

对于在有限时间步内终止的**幕式任务**（比如下棋，有明确的输赢结局），到达终止状态后，所有后续的 n-步回报都等于传统的总回报 G_t。为了分析这种情况，我们可以将公式分成两部分：终止前 $(t+n<T)$ 和终止后：

公式 (10.3) - 幕式任务的展开：

$$g_t^\lambda = (1-\lambda)\sum_{n=1}^{T-t-1}\lambda^{n-1}g_{t:t+n} + \lambda^{T-t-1}G_t$$

两个极端特例验证：
1. **当** $\lambda = 0$ **时**：λ-回报退化为单步回报 $g_{t:t+1}$（即单步 TD）。
2. **当** $\lambda = 1$ **时**：公式前半部分的求和项因为乘了 $(1-1)=0$ 而消失，只剩下最后一项 $1^{T-t-1}G_t = G_t$。这就是标准的**蒙特卡洛方法**。。

离线 λ-回报算法 (Offline λ-return algorithm)

基于上述定义，我们可以定义用于幕式任务学习的**离线 λ-回报算法**。在离线算法中，权重向量在整个回合（episode）进行中是**不更新**的。相反，在回合结束时，会根据半梯度 (semi-gradient) 规则进行一整串的更新，并将 λ-回报作为目标：

公式 (10.4) - 权重更新公式：

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \alpha\left(g_t^\lambda - \hat{v}(s_t,\mathbf{w}_t)\right)\nabla\hat{v}(s_t,\mathbf{w}_t), \quad t = 0,1,\ldots,T-1$$

总而言之，λ-回报为我们提供了一种在**单步 TD** 和**蒙特卡洛方法**之间平滑过渡的新方法。相比于单纯的 n-步 TD，λ-回报算法使用系数 λ 来控制这种平衡，而不是用离散的步数 n。经验表明，λ-回报算法在最佳的参数设置下，表现往往优于其他方法。

- $\mathbf{w}_t$：神经网络的权重（或者是价值表格的值）。
- α：学习率 (Learning rate)。决定了我们每次更新步伐的大小。
- $\left(g_t^\lambda - \hat{v}(s_t,\mathbf{w}_t)\right)$：**TD 误差 (TD Error)**。也就是“我们综合考虑得到的 λ-回报”与“我们原先猜的价值”之间的差值。差值越大，说明我们错得越离谱，越需要大幅更新。
- $\nabla\hat{v}(s_t,\mathbf{w}_t)$：价值函数关于权重 $\mathbf{w}$ 的梯度。它指明了为了修正上面的误差，我们应该把参数 $\mathbf{w}$ 往哪个方向调整。

$TD(\lambda)$ 算法与后向视图

$TD(\lambda)$ 算法简介

$TD(\lambda)$ 是强化学习中最古老、被使用最广泛的算法之一。它是第一个在偏向理论的**前向视图 (forward view)** 和在计算上更友好的**后向视图 (backward view)** 之间建立正式数学关系的算法。

Algorithm 10.1 Semi-gradient TD(λ) for value prediction

```
Initialize weight vector w ∈ ℝ^d arbitrarily
for each episode do
    Draw initial state S
    Let z = 0                                   ▷ z is refreshed for each episode
    repeat                                      ▷ Perform updates at each time step
        Choose an action A ~ π(·|S)
        Take action A, observe R and S'
        z ← γλz + ∇v̂(S, w)
        δ = R + γv̂(S', w) − v̂(S, w)
        w ← w + αδz
        S ← S'
    until S is terminal
end for
return w
```

1. 初始化：

- `Initialize weight vector w`：因为状态太多存不下，我们用一个带权重 $\mathbf{w}$ 的函数（比如神经网络）来逼近价值函数 $\hat{v}(S, \mathbf{w})$。
- `Let z = 0`：初始化资格迹向量为 0。$\mathbf{z}$ 的维度和神经网络的权重 $\mathbf{w}$ 一样大。**它用来记录“哪些权重对最近访问过的状态贡献最大”。**

2. 核心循环（每一步的时间步更新）：

- **计算资格迹：** $\mathbf{z} \leftarrow \gamma\lambda\mathbf{z} + \nabla\hat{v}(S, \mathbf{w})$
  - **推导与含义：** $\nabla\hat{v}(S, \mathbf{w})$ 是当前状态价值关于权重 $\mathbf{w}$ 的梯度。这就好比给当前状态的特征贴上一个“刚刚访问过”的标签。前面的 $\gamma\lambda\mathbf{z}$ 表示随着时间的推移，之前访问过的状态的标签会以 $\gamma\lambda$ 的比例不断衰减。
  - **作用：** 它是 λ^{n-1} 在后向视图中的体现。过去的状态离现在越远，它的迹就越小。
- **计算 TD 误差（单步误差）：** $\delta = R + \gamma\hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$
  - **推导与含义：** 这就是用我们前面提到的**单步回报** $R + \gamma\hat{v}(S', \mathbf{w})$ 作为目标，减去当前的估计值 $\hat{v}(S, \mathbf{w})$。δ 表示我们当前这一步的预测有多准。大于 0 说明比预期的好，小于 0 说明比预期的差。
- **更新权重：** $\mathbf{w} \leftarrow \mathbf{w} + \alpha\delta\mathbf{z}$
  - **推导与含义：** 传统的梯度下降更新公式是 $\mathbf{w} \leftarrow \mathbf{w} + \alpha\delta\nabla\hat{v}(S, \mathbf{w})$（只更新当前状态对应的权重）。
  - **灵魂所在：** 这里把当前状态的梯度 $\nabla\hat{v}$ 换成了资格迹 $\mathbf{z}$。这意味着，一旦当前这

一步产生了误差 δ，我们**不仅要更新当前状态的权重，还要沿着记忆（资格迹 $\mathbf{z}$）顺藤摸瓜，把刚刚经历过的所有状态的权重都更新一遍！** 离得越近的状态（$\mathbf{z}$ 值越大），更新幅度越大。

总结一下这个算法的妙处：

它不需要等回合结束，每走一步就能计算出一个单步误差 δ，然后把这个误差"广播"给所有过去留有"资格迹"的状态。这完美地用代码实现了一开始那个需要看透未来 λ^{n-1} 的理论公式。

资格迹 (Eligibility Trace) $\mathbf{z}_t$

$TD(\lambda)$ 算法基于一个新概念：**资格迹 (eligibility trace)**。它是一个维度与权重向量 $\mathbf{w}_t$ 相同的向量 $\mathbf{z}_t \in \mathbb{R}^d$。

• $\mathbf{w}_t$ **(权重向量)**：可以被视为一种**长期记忆 (long-term memory)**，它在系统的整个生命周期中积累信息。

• $\mathbf{z}_t$ **(资格迹)**：是一种**短期记忆 (short-term memory)**，它只保存临时的信息，持续时间不到一个回合 (episode)。

公式 (10.5) - 资格迹的初始化与增量更新：

$$\mathbf{z}_{-1} = \mathbf{0}, \quad \text{且} \quad \mathbf{z}_t = \gamma\lambda\mathbf{z}_{t-1} + \nabla\hat{v}(s_t, \mathbf{w}_t)$$

- $\mathbf{z}_t$：时间步 t 的资格迹向量。记录了哪些状态/特征对当前的局面"有责任"。
- γ：折扣率 (discount rate)。
- λ：迹衰减参数 (trace-decay parameter)。$\gamma\lambda$ 共同决定了过去状态的责任随时间衰减的速度。
- $\nabla\hat{v}(s_t, \mathbf{w}_t)$：当前状态价值估计对权重的**梯度**。代表当前状态的特征。

随着时间流逝，旧的记忆（$t-1$ 时刻以前的足迹）会以 $\gamma\lambda$ 的速度慢慢消失。离当前越远的状态，分到的功劳就越少。

目标：我们希望当前的误差 δ_t（TD error）不仅能更新当前状态，还能按比例更新之前的状态。**权重分配**：根据 λ-回报的定义， k 步前的状态对当前误差的责任应该是 $(\gamma\lambda)^k$。**递归构造**：

• $t=0$ 时：$\mathbf{z}_0 = \nabla\hat{v}_0$

• $t=1$ 时：我们要保留 0 时刻的记忆（衰减一下），加上 1 时刻的新印象：$\mathbf{z}_1 =$

$(\gamma\lambda)\mathbf{z}_0 + \nabla\hat{v}_1$
• $t = 2$ 时：$\mathbf{z}_2 = (\gamma\lambda)\mathbf{z}_1 + \nabla\hat{v}_2$

结果：展开这个递归式，你会发现 $\mathbf{z}_t$ 实际上自动包含了之前所有梯度的**加权和**：

$\mathbf{z}_t = \sum_{k=0}^{t}(\gamma\lambda)^{t-k}\nabla\hat{v}(s_k, \mathbf{w}_k)$

通俗举例解释：长期记忆 vs 短期记忆
想象你正在训练一只小狗（Agent）走迷宫：

- **长期记忆 ($\mathbf{w}_t$)**：小狗大脑里形成的“迷宫整体地图”。这是经过无数次尝试慢慢建立起来的。
- **短期记忆/资格迹 ($\mathbf{z}_t$)**：小狗在当前这**一次**尝试中，沿途留下的“气味标记”。每经过一个路口（状态），它就留下气味（加上 $\nabla\hat{v}$）。随着时间推移，早先路口的气味会变淡（乘以衰减系数 $\gamma\lambda$）。如果最终在终点找到了骨头，小狗会顺着未消散的气味往回找，气味越浓的路口（近期经过的），被认为对找到骨头功劳越大，其“地图印象（$\mathbf{w}$）”就被强化得越多。

TD 误差与权重更新

资格迹 $\mathbf{z}_t$ 在一个回合中不断累积价值估计的梯度，并用随时间指数衰减的**新近度系数 (recency coefficients)** $\gamma\lambda$ 进行加权。
在 $TD(\lambda)$ 中，权重向量在每个时间步都会进行更新，更新的幅度取决于**TD 误差**乘以**资格迹**。
公式 (10.6) - TD 误差：

$$\delta_t = r_{t+1} + \gamma\hat{v}(s_{t+1}, \mathbf{w}_t) - \hat{v}(s_t, \mathbf{w}_t)$$

公式 (10.7) - 权重更新：

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \alpha\delta_t\mathbf{z}_t$$

- δ_t：TD误差。代表“实际发生的事情（真实奖励+下一步的预期）”与“我们原先对当前状态的预期”之间的**偏差**。如果是正数，说明比预期的好（有惊喜）；负数则说明比预期的差（很失望）。
- α：学习率。每次参数调整的步伐大小。

后向视图 (Backward View) 与 两个特例

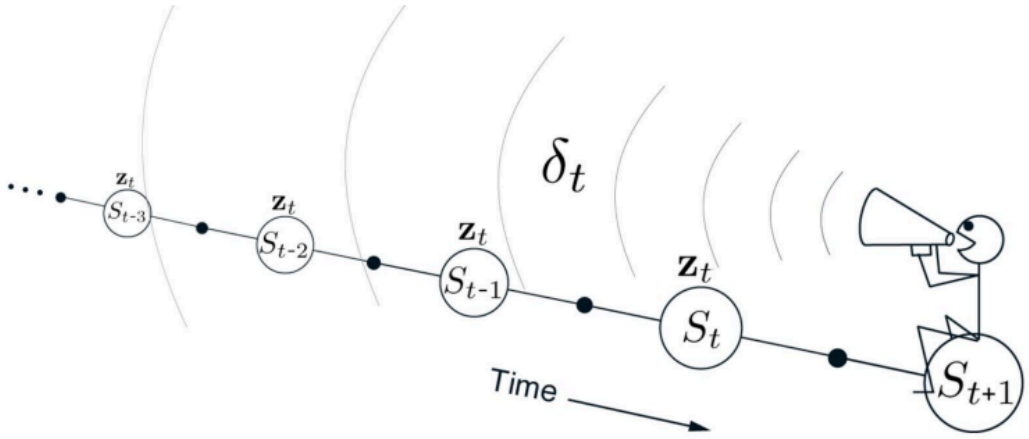

Figure 10.2: The backward view of TD(λ). Each update depends on the current TD error combined with the current eligibility trace of past events.

我们注意到 $TD(\lambda)$ 在时间上是**向后看的 (oriented backward)**。在每一步，它都会计算当前的 TD 误差 δ_t，并将其“向后分配”给之前经历过的每个状态。分配的多少取决于那个状态对当前**资格迹**的贡献程度（如图 10.2 所示：小人在时刻 $t+1$ 拿着大喇叭把当前的误差 δ_t 向过去的历史状态喊话，离得越远的状态听到的声音越小）。
两个特殊情况：、

- **当 $\lambda = 0$ 时 ($TD(0)$)**：此时 $\mathbf{z}_t = \nabla \hat{v}(s_t, \mathbf{w}_t)$。资格迹瞬间清空过去的所有记忆，只包含当前状态的梯度。这退化成了**单步半梯度 TD 更新**。
- **当 $\lambda = 1$ 时 ($TD(1)$)**：赋予早期状态的信用只按照 γ 衰减（没有被 λ 额外截断）。这实际上就是**蒙特卡洛 (Monte Carlo)** 方法的在线增量实现方式。
- **为什么 $TD(\lambda)$ 比离线的 λ-回报算法更好？（三大优势）**
  1. **步步更新**：它在回合的**每一个时间步**都会更新权重向量，而不是干等到整个回合结束（回合没结束就能开始学习纠错）。
  2. **计算平摊**：它的计算量在时间上是均匀分布的（每步算一点），而不是在回合结束时进行一次庞大的计算。
  3. **适用性更广**：它可以很容易地应用于**持续性问题 (continuing problems)**（比如机器人维持平衡，永远没有明确的“回合结束”），而不仅仅局限于幕式任务（episodic ones）。

n步截断 λ-回报与在线算法

理论上，**离线 λ-回报算法**很重要，但在实践中却不太行得通。因为你必须等到整个回合 (episode) 结束，才能知道真正的 λ-回报 g_t^λ 是多少。如果遇到持续性问题（没有终点），你永远也算不出精确的 g_t^λ。

截断 λ-回报的定义 (Definition of truncated λ-return)

不过，值得注意的是，算法对"很久以后才发生的奖励"的依赖其实是很弱的，因为权重会随着时间指数级衰减 (λ^{n-1})。

因此，一个自然的近似方法是：**在往未来看几步之后，直接把序列"截断"**。基于这个想法，我们定义了给定到未来视野 (horizon) h 的数据时，时刻 t 的**截断 λ-回报**：

公式 (10.8) - 截断 λ-回报：

$$g_{t:h}^\lambda \triangleq (1-\lambda)\sum_{n=1}^{h-t-1}\lambda^{n-1}g_{t:t+n} + \lambda^{h-t-1}g_{t:h}, \quad 0 \le t < h \le T.$$

(注意：这个公式和前一节的 (10.3) 几乎一样，只是把终点 T 替换成了视野 h，把总回报 G_t 替换成了目前能拿到的最长回报 $g_{t:h}$)

公式 (10.9) - 截断算法更新规则：

$$\mathbf{w}_{t+n} \leftarrow \mathbf{w}_{t+n-1} + \alpha\left(g_{t:t+n}^\lambda - \hat{v}(s_t, \mathbf{w}_{t+n-1})\right)\nabla\hat{v}(s_t, \mathbf{w}_{t+n-1})$$

基于此，我们得到了一系列算法，在状态价值估计中，被称为 **截断 TD(λ)**，或 **TTD(λ)**。这种方法的更新会延迟 n 步。

为什么要截断？

想象你在预测一家公司的股票价值。

- **离线 λ-回报**：要求你必须等到这家公司倒闭（回合结束 T），清算完所有资产，才能回头准确评估它今天的价值。这太扯了。
- **截断 λ-回报**：由于通货膨胀和市场变数（衰减系数 λ），100 年后的收益对今天估值的影响微乎其微。所以我们设定一个**视野 (horizon, h)**，比如只往后看 3 年。把 3 年后的估值当作最终结果直接截断，不去管 4 年后的事。这在工程上非常实用。

在线 λ-回报算法 (Online λ-return algorithm) 的推导

在截断 TD(λ) 中选择截断参数 n 需要权衡：n 太大，更新太慢（得等很久）；n 太小，近似效果差。

为了"全都要"，我们推导出了**在线 λ-回报算法**。代价是计算复杂度大幅上升。

核心机制： 该算法在**每一个时间步**，都会根据新收集到的数据，**从头（回合第一步）开始重新计算一遍所有的更新**。因为包含了未来的新数据，这些"重新计算"的更新比旧的更新更精确。

算法流程漫游 (Walkthrough) 与通俗解析
让我们跟着原文，看看当视野扩展到 $h = 1, 2, 3$ 时，算法在干嘛。
初始状态： 回合在时间 0 开始，初始状态 s_0，初始权重 $\mathbf{w}_0$。
1. **视野 $h = 1$ (走完了第1步)：**
   - 我们看到了第一步的奖励 r_1 和新状态 s_1。
   - 计算目标 $g_{0:1}^{\lambda}$。
   - **更新：** 用公式 (10.10) 把 $\mathbf{w}_0$ 更新为 $\mathbf{w}_1$。
2. **视野 $h = 2$ (走完了第2步，关键来了！)：**
   - 我们看到了 r_2 和 s_2。**我们有了新情报！**
   - 我们不仅计算第 1 步的回报 $g_{1:2}^{\lambda}$，我们还要**回过头去**，利用新情报重新计算第 0 步的回报 $g_{0:2}^{\lambda}$（此时第 0 步的目标比刚才 $h = 1$ 时更准了）。
   - **重做更新：**
     - 公式 (10.11)：带着更准的 $g_{0:2}^{\lambda}$，从最初的 $\mathbf{w}_0$ 重新算出新的 $\mathbf{w}_1$。
     - 公式 (10.12)：基于刚才重新算出的 $\mathbf{w}_1$，加上目标 $g_{1:2}^{\lambda}$，算出 $\mathbf{w}_2$。
3. **视野 $h = 3$ (走完了第3步)：**
   - 看到 r_3 和 s_3。
   - 计算包含了 3 步远见的最新目标：$g_{0:3}^{\lambda}, g_{1:3}^{\lambda}, g_{2:3}^{\lambda}$。
   - **再次重做更新：** (公式 10.13 - 10.15)
     - 从 $\mathbf{w}_0$ 出发，用最新最准的目标重写 $\mathbf{w}_1$。
     - 用重写后的 $\mathbf{w}_1$，重写 $\mathbf{w}_2$。
     - 用重写后的 $\mathbf{w}_2$，计算得出 $\mathbf{w}_3$。

💡 **通俗举例解释：什么是“重做更新 (Redo updates)”？**
想象你在**连载写小说**：
• **第1天 ($h = 1$)**：你写了第一章，主角捡到一把剑（更新 $\mathbf{w}_1$）。
• **第2天 ($h = 2$)**：你写第二章，突然灵感爆发，决定设定这把剑里封印了魔王。为了逻辑严密，你**回退到第一章**（重做 $\mathbf{w}_1$），加上一段“剑身散发诡异黑气”的伏笔。然后顺理成章地写完第二章（更新 $\mathbf{w}_2$）。
• **第3天 ($h = 3$)**：写第三章时，你发现魔王其实是主角的父亲。于是你**再次回退**，把第一章的伏笔修改为“剑柄刻着主角家族的家徽”（再次重做 $\mathbf{w}_1$），修改第二章魔王苏醒时的台词（重做 $\mathbf{w}_2$），最后写出第三章父子相认（更新 $\mathbf{w}_3$）。
在线 λ-回报算法，就是一个不断利用“事后诸葛亮”的最新情报，回过头去疯狂修改历史认知（权重），从而得到当前最完美权重的算法。

统一符号与总结

因为我们每一轮都在生成不同的权重序列（但都是从同一个 $\mathbf{w}_0$ 开始），为了统一表示，我们引入了上标符号 $\mathbf{w}_t^h$：
• 它表示：在**视野扩展到 h** 的那一轮遍历中，时间步 t 更新后的权重。

使用这个符号，上面繁琐的 $h = 1, 2, 3$ 的公式 (10.16 到 10.21) 可以被极其简洁地归纳为一个统一的通用公式：

公式 (10.22) - 在线 λ-回报算法的统一更新规则：

$$\mathbf{w}_{t+1}^h \leftarrow \mathbf{w}_t^h + \alpha \left(g_{t:h}^\lambda - \hat{v}(s_t, \mathbf{w}_t^h) \right) \nabla \hat{v}(s_t, \mathbf{w}_t^h), \quad 0 \le t < h \le T.$$

符号着重解释

• $\mathbf{w}_t^h$：理解为 **"站在上帝视角 h 的时间点，去重新审视并确定的时刻 t 的参数"**。

• $\mathbf{w}_{t+1}^h$：在相同的上帝视角 h 下，计算出来的下一个时刻 $t+1$ 的参数。

优缺点总结

• **优点**：完全是在线的 (fully online)，每走一步都能利用当前所有已知信息，榨干数据的价值，精度极高。

• **致命缺点**：**计算复杂度高**。如原文最后一句所说，随着视野 h 的增加，计算量呈**二次方 (quadratically)** 增长。每走一步，都要把之前所有的步数重新算一遍，如果一局游戏有 1000 步，走到第 1000 步时你要从头重算 1000 次，这在算力上是灾难性的。

Sarsa(λ) 及其变体

从"状态"到"动作"的跨越 (动作价值的资格迹)

在本节中，我们将推导一种基于**资格迹 (eligibility traces)** 的**控制算法 (control algorithm)**。为了达到这个目的，我们首先需要将资格迹的概念从"状态价值 (state values)"扩展到**"动作价值 (action values)"**。

我们需要重新定义动作价值形式的 n**-步回报**：

公式 (10.23) - 动作价值的 n-步回报：

$$g_{t:t+n} \triangleq r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^{n-1} r_{t+n} + \gamma^n \hat{q}(s_{t+n}, a_{t+n}, \mathbf{w}_{t+n-1})$$

(当 $t + n \ge T$ 时，$g_{t:t+n} = g_t$)

利用这个公式，我们可以像之前一样，构建用于动作价值的**截断 λ-回报**：

公式 (10.24) - 动作价值的截断 λ-回报：

$$g_{t:h}^\lambda = (1-\lambda) \sum_{n=1}^{h-t-1} \lambda^{n-1} g_{t:t+n} + \lambda^{h-t-1} g_{t:h}, \quad 0 \le t < h \le T.$$

这与状态价值的截断 λ-回报完全相同，唯一的区别是这里的 $g_{t:h}$ 使用了动作价值的形式。

- $\hat{q}(s, a, \mathbf{w})$：**动作价值函数 (Action-value function)**。以前我们用 $\hat{v}(s, \mathbf{w})$ 评估"处于状态 s 有多好"，现在我们用 $\hat{q}$ 评估"在状态 s 下，执行动作 a 有多好"。

这是能够直接指导 Agent 做出决策的关键。

- **通俗举例解释：为什么要换成“动作价值”？**
  - **状态价值 ($\hat{v}$)**：就像你站在一个十字路口，有人告诉你“这个路口风水很好，是个好地方”。但这不能帮你决定是该左转还是右转。
  - **动作价值 ($\hat{q}$)**：不仅告诉你路口的情况，还明确标出“在这个路口**左转**能捡到 100 块钱，**右转**会掉进坑里”。Sarsa(λ) 的目标就是学习这个 $\hat{q}$ 值，这样 Agent 只要每次选择 $\hat{q}$ 值最高的动作，就能实现通关。

离线 λ-回报算法 (动作价值版)

基于上面的定义，我们很容易推导出动作价值的**离线 λ-回报算法**，其更新规则如下：
公式 (10.25) - 离线更新规则：

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \alpha \left(g_t^\lambda - \hat{q}(s_t, a_t, \mathbf{w}_t)\right) \nabla \hat{q}(s_t, a_t, \mathbf{w}_t), \quad t = 0, \ldots, T-1$$

(注意：这里 $g_t^\lambda \triangleq g_{t:\infty}^\lambda$。对于离线算法，更新必须在回合结束时进行，因为那时我们才能收集到构建 g_t^λ 所需的所有信息。)

Sarsa(λ) 算法核心机制

针对动作价值的时序差分方法被称为 **Sarsa(λ)**。它通过一个简化的更新规则来近似前向视图 (forward view)：
公式 (10.26) - Sarsa(λ) 权重更新：

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \alpha \delta_t \mathbf{z}_t$$

其中 δ_t 是 **TD 误差**：
公式 (10.27) - 动作价值的 TD 误差：

$$\delta_t \triangleq r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, \mathbf{w}_t) - \hat{q}(s_t, a_t, \mathbf{w}_t)$$

动作价值形式的**资格迹**定义如下：
公式 (10.28) - 资格迹的更新：

$$\mathbf{z}_{-1} = \mathbf{0}, \quad \mathbf{z}_t = \gamma \lambda \mathbf{z}_{t-1} + \nabla \hat{q}(s_t, a_t, \mathbf{w}_t), \quad 0 \le t \le T.$$

Sarsa(λ) 的伪代码如算法 10.2 所示。

算法 10.2 流程拆解与通俗举例

Algorithm 10.2 Semi-gradient Sarsa(λ) algorithm

```
Initialize weight vector w ∈ ℝ^d arbitrarily
for each episode do
    Draw initial state s and action a ~ π(·|s)
    Let z = 0                                        ▷ z is refreshed for each episode
    repeat                                           ▷ Perform updates at each time step
        Take action a, observe r and s'
        if S' is not terminal then
            Choose action a' ~ π(·|s')
            z ← γλz + ∇q̂(s, a, w)
            δ = r + γq̂(s', a', w) − q̂(s, a, w)
            w ← w + αδz
            a ← a'
        end if
        s ← s'
    until s is terminal
end for
return w
```

Sarsa(λ) 的名字来源于它更新一次所需的数据序列：**S**tate (当前状态) → **A**ction (当前动作) → **R**eward (奖励) → **S**tate' (下一步状态) → **A**ction' (下一步动作)。

通俗场景还原 (如何利用资格迹 $\mathbf{z}_t$ 分配责任)：
假设你在玩超级马里奥（Agent）。
1. **(S, A)**：你在悬崖边 (状态 s)，决定按跳跃键 (动作 a)。
2. **(R, S')**：你跳到了空中 (新状态 s')，暂时没死也没有得分 (奖励 $r = 0$)。
3. **(A')**：在空中时，你决定按“向下冲刺”键 (新动作 a')。
4. **计算 TD 误差 δ**：你比较了一下“跳跃后接向下冲刺”的预期价值，和“当初决定跳跃”的预期价值。如果发现预期变差了（比如下面是岩浆），TD 误差 δ 就是负的（代表“糟了！”）。
5. **资格迹 $\mathbf{z}$ 的作用**：这个“糟了”的惩罚应该扣在谁头上？资格迹 $\mathbf{z}$ 记录了你之前的操作历史。它会把惩罚**主要扣在刚刚按的“向下冲刺”上**，同时也会**稍微扣一点在之前的“跳跃”上**（因为 $\gamma\lambda$ 的衰减）。
6. **更新 $\mathbf{w}$**：根据这个惩罚力度，调整大脑神经网络的权重 $\mathbf{w}$。下次再遇到悬崖，你跳跃或向下冲刺的概率就会降低。

在线 λ-回报算法 (动作价值版)

动作价值也存在其对应的**在线 λ-回报算法**，以及在具有线性函数近似设置下极其高效的实现方式，称为 **true online TD(λ)**（真正的在线 TD 算法）。
这些算法与之前为“价值预测”设计的原始算法几乎一模一样！唯一的区别在于特征向

量的设计：
• 以前（状态价值）：输入特征只和状态有关，即 $\mathbf{x}_t = \mathbf{x}(s_t)$。
• 现在（动作价值）：输入特征同时考虑了状态和动作的组合，变成了 $\mathbf{x}_t = \mathbf{x}(s_t, a_t)$
。

变量参数 与 10.6 树回溯(λ)

变量 λ 与 γ (不再是固定常数)

对于前面介绍的所有算法，$TD(\lambda)$、Sarsa(λ) 等，它们都由两个重要的参数控制：

- λ 控制**自举 (bootstrapping)** 的程度（即我们多大程度上依赖自己的“猜测”）
- γ 控制**折扣 (discounting)**（即未来奖励在今天的价值衰减）。
  将这些参数从“常数”泛化为“依赖于状态和动作的函数”是非常有用的。具体来说，我们可以定义

  $\lambda_t \triangleq \lambda(s_t, a_t)$ 和 $\gamma_t \triangleq \gamma(s_t)$。
- **公式 (10.29) - 泛化的目标回报：**

  $g_t = \sum_{n=1}^{\infty} r_{t+n} \prod_{i=1}^{n-1} \gamma_{t+i}$
  （当 γ 不是常数时，我们不能简单地写 γ^{n-1}，而是要把沿途每一步的 γ_t 乘起来。）|

  通俗解析：依赖状态的终止 (State-dependent termination)

  以前我们总是把任务分为两类：“幕式任务（有游戏结束画面）”和“持续任务（永远不结束）”。
  把 γ 变成状态的函数后，我们可以**用一种数学方式把这两者统一起来**！

  怎么做？ 只要定义在“游戏结束”的状态下，$\gamma(s) = 0$ 即可。

  为什么奇妙？ 因为一旦 $\gamma = 0$，意味着未来无论拿多少分，乘上 0 之后对现在都没有影响了（相当于未来的奖励被100%没收）。这使得我们不需要在代码里写特殊的 `if terminal:` 分支，整个过程可以看作是一条永远在进行的无限长数据流。

  递归形式的 λ-回报 (公式 10.30 - 10.32)
  为了把公式写得更优雅，作者将 λ-回报重写成了**递归形式**：

  公式 (10.31) - 状态价值的递归 λ-回报：

  $g_t^{\lambda s} = R_{t+1} + \gamma_{t+1} \left((1 - \lambda_{t+1}) \hat{v}(s_{t+1}, \mathbf{w}_t) + \lambda_{t+1} g_{t+1}^{\lambda s} \right)$

一句话看懂： “今天的终极目标值” = “明天的真实奖励” + “明天的折扣” × [(部分权重) 看明天的价值预测 + (部分权重) 继续往后推演明天的终极目标值]。

树回溯(λ) (Tree-Backup(λ) / $TB(\lambda)$)

之前提到的 Sarsa 算法强依赖于**下一步实际采取的动作** a_{t+1}。如果你的策略包含随机探索（比如经常随机乱走），这会导致学习过程中**方差很大**（每一步的估值忽高忽低）。

树回溯 (Tree-backup) 算法提供了一种替代方案：它在每一步都会把**所有可能的动作**都考虑进去，代价是增加了计算量。结合了资格迹后，我们称之为 **Tree-Backup(λ)** 或 $TB(\lambda)$。

图 10.3：树回溯的结构图解 (结合公式 10.33 与 10.34)

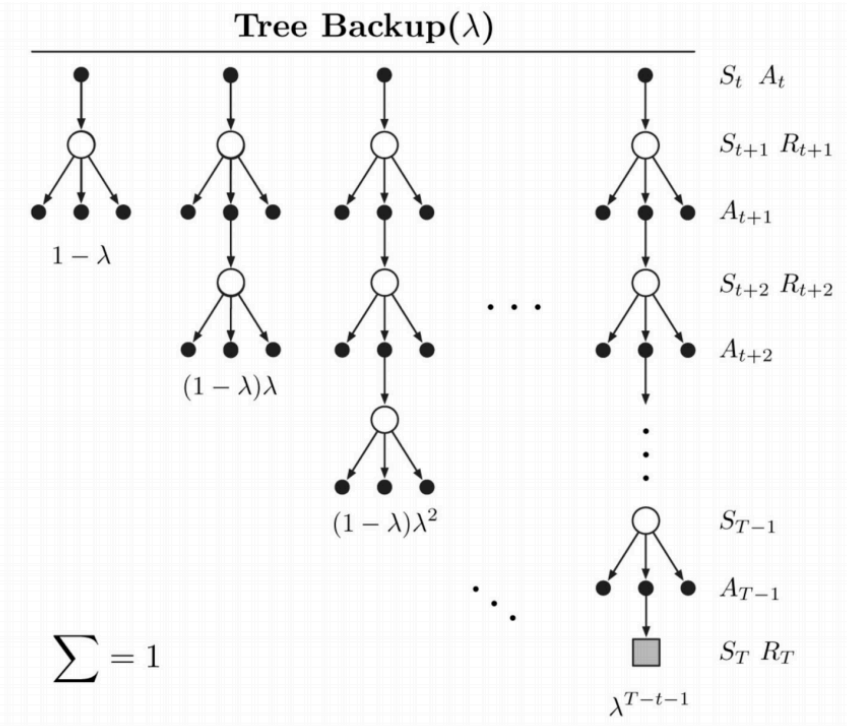

Figure 10.3: The backup diagram of *Tree-Backup(λ)*.

- **中间粗壮的黑色主干**：这是 Agent **实际**走过的轨迹 ($S_t \to A_t \to S_{t+1} \ldots$)。
- **旁边的空心小圆圈 (分支)**：这是在每个状态下，Agent **没选**的动作。

- **$TB(\lambda)$ 是怎么计算目标的 (公式 10.33)：**

$$g_t^{\lambda a} = r_{t+1} + \gamma_{t+1}\Big[(1-\lambda_{t+1})\bar{V}_t(s_{t+1}) + \lambda_{t+1}\Big(\sum_{a\neq a_{t+1}} \pi(a|s_{t+1})\hat{q}(s_{t+1},a,\mathbf{w}_t) + \pi(a_{t+1}|s_{t+1})g_{t+1}^{\lambda a}\Big)\Big], \quad (10.33)$$

where

$$\bar{V}_t(s_{t+1}) = \sum_a \pi(a|s_{t+1})\hat{q}(s_{t+1},a,\mathbf{w}_t). \quad (10.34)$$

它不像 Sarsa 只盯着实际走的那条路。在每个节点，它会“环顾四周”：
1. 对于**没选的动作**：直接用当前的价值网络去评估它们，乘以它们被选中的概率 $\pi(a|s_{t+1})$。
2. 对于**实际选中的动作**：顺着主干继续往下深入（使用资格迹 λ 向下递归 $g_{t+1}^{\lambda a}$）。

通俗举例解释：Sarsa vs 树回溯

- **Sarsa (单线侦察兵)**：队长派侦察兵去探路。侦察兵到了一个十字路口，抛硬币决定向左走。他回来报告：“报告队长，我向左走了，遇到了一条狗，路况不好。” 队长只能根据“向左走遇到狗”来更新整个十字路口的评价。如果下次向右走其实有金币，Sarsa 这一次根本学不到。
- **Tree-Backup (带无人机的侦察兵)**：侦察兵带了无人机。到了十字路口，他虽然还是决定向左走（主干），但他升起无人机看了一眼向右的路（分支），发现那边有金币！他回来报告：“报告队长，我向左走遇到了狗，但是根据无人机观测，向右走有 80% 概率拿到金币。” 队长据此更新评价，估值会更平稳、更全面。

$TB(\lambda)$ 的资格迹更新 (公式 10.35)

这是整个 $TB(\lambda)$ 的灵魂所在：

公式 (10.35) - 树回溯的资格迹更新：

$\mathbf{z}_t \leftarrow \gamma_t\lambda_t\pi(a_t|s_t)\mathbf{z}_{t-1} + \nabla\hat{q}(s_t,a_t,\mathbf{w})$

重点看衰减项： $\gamma_t\lambda_t\pi(a_t|s_t)\mathbf{z}_{t-1}$

这里多乘了一个 $\pi(a_t|s_t)$ **(即目标策略选择实际动作的概率)**。

为什么要乘以这个概率？

想象 Agent 实际走了一条非常离谱的路线（比如误触了万分之一概率的自毁按钮）。

如果你用普通的 λ 算法，这个严重的后果会顺着资格迹一直往回传，导致前面所有正常的决策都被连累扣分。

但在 $TB(\lambda)$ 中，如果实际采取的动作 a_t 在目标策略中的概率 $\pi(a_t|s_t)$ 非常低（比如接近 0），那么这一步的资格迹就会被瞬间**“掐断” (乘以 0)！

这意味着：“如果你做了一个不符合常规策略的罕见动作，那么之后的后果就不再追溯给之前的状态了。"** 这就是 $TB(\lambda)$ 能够在**离策略 (off-policy)** 情况下安全学习，而不需要使用复杂的“重要性采样 (Importance Sampling)”技术的根本原因！